

# **FJ1 ASSIGNMENT**

**Subject:** Real-Time Embedded System  
(21EIC504J)

**Title:** Memory System Mechanisms and Their  
Impact on Real Time Determinism in ARM  
Based Cyber Physical Robotic Systems

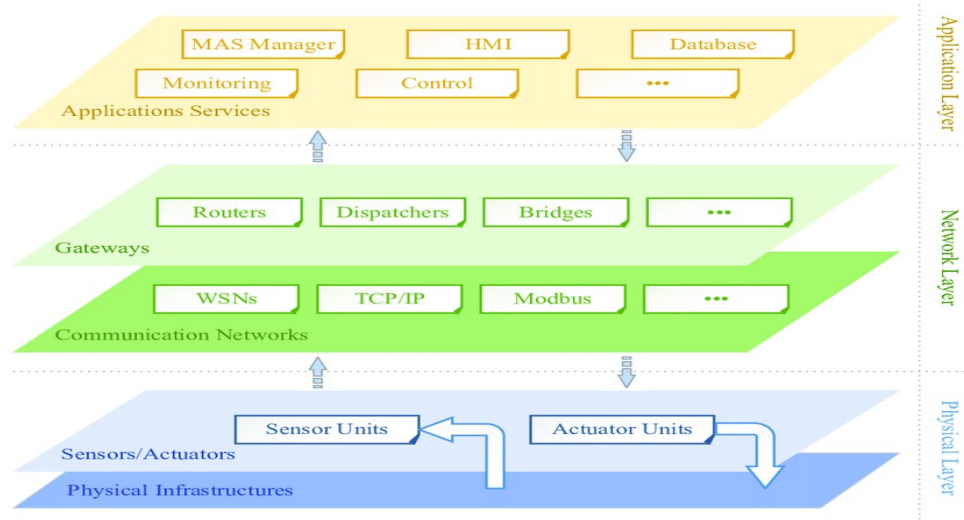
**Submitted by:**

Bharathwaj Manoharan  
(RA2512053010003)

# 1. Introduction & Background

Cyber physical systems CPS are integrated assemblies of computation, networking, and physical processes in which embedded controllers monitor and actuate physical plants in closed loop operation. In robotics, CPS manifests as autonomous or semi-autonomous agents that couple sensors, actuators, and real time controllers to meet functional and safety requirements. Real time determinism in such systems denotes the property that temporal behavior, specifically task response times and deadlines, is bounded and predictable under all specified operating conditions. Worst case execution time WCET is the principal metric used to certify such temporal bounds; it is the maximal execution time a task may require on a given hardware and software platform under specified assumptions.

ARM based embedded processors dominate many robotic platforms because they provide a favorable balance of performance, energy consumption, and ecosystem support. Architectures within the ARM family range from simple microcontroller class Cortex M cores often without MMU and sometimes with MPU, to application processors Cortex A with multi-level caches, MMU, out of order engines and rich interconnects. These microarchitectural features that accelerate typical workloads also introduce dynamic timing variability that complicates WCET estimation and undermines hard real time guarantees if not managed explicitly.



**Figure:** Cyber Physical Robotic System with Memory Hierarchy.

Memory hierarchy mechanisms comprising processor caches L1 L2 L3, translation lookaside buffers TLBs, the memory management unit MMU or memory protection unit MPU, DRAM organization, and coherence protocols substantially influence both average performance and worst-case timing. Caches reduce average memory latency but make access latency data dependent; address translation TLB hits and misses and page walks and cache coherence events introduce additional, often non local and system level delays. This assignment critically examines how cache architecture, memory hierarchy, MMU and MPU use, and address translation affect real time determinism in embedded robotic applications, evaluates tradeoffs between performance and predictability, quantifies effects with cycle level examples, and proposes architectural strategies to balance high performance with deterministic behavior.

The central problem is a tradeoff between performance throughput and energy efficiency and determinism tight, analyzable WCET. High performance features such as deep cache hierarchies, speculative execution, coherent multi core interconnects, and virtual memory improve average throughput but increase worst case variability; conversely, removing or constraining these features yields predictability at the cost of poorer average performance or flexibility. The remainder of this document develops the evidence and analytic framework required to reason about these tradeoffs in ARM based robotic controllers.

## **2. Literature Review**

Cache behavior in real time systems has been extensively studied because caches are a primary source of temporal unpredictability. Research surveys consolidate static and measurement-based techniques for cache analysis and management. Static cache analysis classifies memory references as must hit, must miss, or unknown and integrates these classifications into WCET computation; its conservativeness often produces pessimistic WCETs but provides safety guarantees. Measurement based approaches complement static methods by empirically bounding execution times but require careful environment control to remain sound.

WCET analysis challenges become acute when MMUs and virtual memory are present. Address translation introduces two main difficulties.

First, TLB misses can trigger page table walks whose latency depends on page table residency and cache state. Second, virtual memory enables aliasing and dynamic mapping that complicates cache indexing and associativity reasoning, hampering static set classification analyses. Time predictable virtual memory proposals and TLB locking mitigate some effects by restricting dynamic behavior, but they trade flexibility for analyzability.

Multicore interference is a more recent and pressing research area for CPS platforms that adopt multi core ARM application processors. Shared resources such as last level caches, DRAM banks, and interconnects create interference channels that multiply worst case latencies beyond uniprocessor estimates. Mitigation approaches can be categorized as isolation techniques such as partitioning or way locking caches, scheduling techniques such as spatial and temporal partitioning, hardware quality of service support, and deterministic memory hierarchies using scratchpad and bank partitioning. Studies show that naive partitioning can be non-monotonic with respect to WCET; cache size increases do not always reduce worst case cost because of miss pattern changes.

Cache partitioning and coloring reduce inter task interference and can improve schedulability, but static partitioning may underutilize resources and can be sensitive to task working set changes. Cache locking and way locking provide tight predictability for critical code sections but require careful allocation and may impede dynamic behaviors such as online updates. Scratchpad memories replace caches for critical code and data and provide deterministic access times when managed statically; however, they introduce programming complexity and may not scale for large software stacks. Virtual memory solutions that constrain translation dynamics strike a middle ground by limiting unpredictability while retaining some flexibility, yet they require operating system support and increase implementation complexity.

Overall, research indicates that no single technique universally resolves the performance predictability trade off. Compositional methods combining hardware support partitioning and locking, software techniques static allocation and WCET directed mapping, and runtime orchestration scheduling and bandwidth enforcement are necessary for embedded CPS.

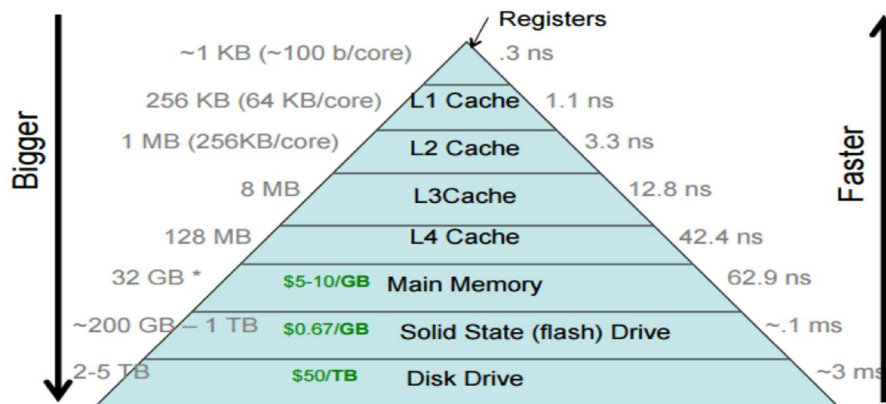
### 3. Technical Foundations and Conceptual Framework

#### a. Cache Architecture Fundamentals

A cache is a small, fast memory placed between the CPU and the next memory level to exploit temporal and spatial locality. Main cache design parameters include:

- Capacity (C): total cache size in bytes.
- Line (block) size (B): number of bytes per cache line.
- Associativity (A): number of ways (direct-mapped:  $A=1$ ; fully associative:  $A=C/B$ ).
- Number of sets (S):  $S = C/(B.A)$ .
- Replacement policy: LRU, pseudo-LRU, random, FIFO.
- Write policy: write-back vs write-through; write allocate vs no-write allocate.

Associativity and replacement policy determine which memory addresses map to each set and how eviction occurs. These parameters are central to static cache analysis and WCET modeling.



**Figure:** Cache memory hierarchy (L1, L2, L3) with hit/miss flow.

## b. Average Memory Access Time and Worst-Case Latency Modeling

Average Memory Access Time (AMAT) for a two-level cache can be expressed as:

$$AMAT = T_{L1} + MR_{L1} \cdot T_{miss-L1}$$

Where  $T_{L1}$  is L1 hit time,  $MR_{L1}$  is L1 miss rate, and  $T_{miss-L1}$  is miss penalty (time to service from L2/DRAM). For multi-level hierarchies, recursively:

$$AMAT = T_{L1} + MR_{L1} \cdot (T_{L2} + MR_{L2} \cdot T_{miss-L2})$$

Worst-case memory access time (WC-MAT) replaces average miss rates with worst-case miss counts or rates:

$$WC\_MAT = T_{L1} + MR_{L1}^{wc} \cdot (T_{L2} + MR_{L2}^{wc} \cdot T_{DRAM})$$

WCET of a task, in a simplified instruction-plus-memory model, can be expressed:

$$WCET = C_{CPU} + N_{mem} \cdot WC\_MAT + I$$

Where  $C_{CPU}$  is CPU cycles for non-memory instructions,  $N_{mem}$  is number of memory references on the WCET path, and  $I$  is external interference (e.g., coherence invalidations, DRAM contention)

## c. TLB Page Tables and Address Translation Pipeline

An MMU translates virtual addresses (VA) to physical addresses (PA) via multi-level page tables. A TLB caches recent translations. Translation sequence:

- CPU issues VA.
- TLB lookup (associative): if hit  $\rightarrow$  use PA (cost  $T_{TLB\_hit}$ ).
- If miss  $\rightarrow$  page table walk (cost  $T_{PTW}$ , possibly multiple memory accesses and cache effects) and TLB refill.

- TLB refill may require eviction and invalidate entries in other cores (coherence for TLB shootdowns).

Effective translation latency:

$$T_{trans} = T_{TLB\_hi} + P_{TLB\_miss} \cdot T_{PTW}$$

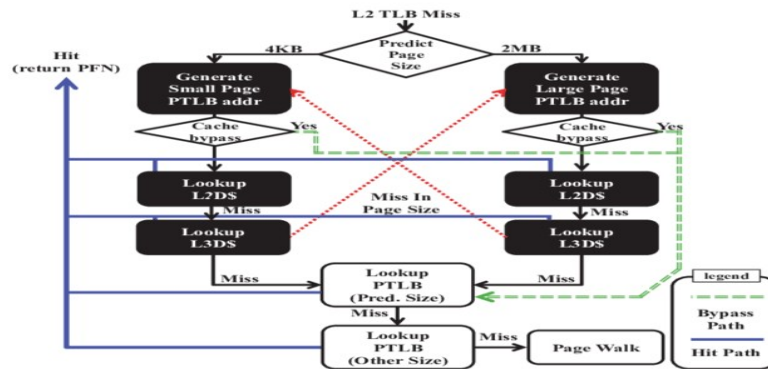
Where  $P_{TLB\_miss}$  is TLB miss probability. Page walks may themselves generate cache misses and additional DRAM accesses and therefore are not constant-time unless TLB/page tables are locked.

## d. Cache Coherence Protocols

In multicore systems, coherence protocols (MESI, MOESI, etc.) maintain a coherent view of memory by tracking cache lines' states (Modified, Exclusive, Shared, Invalid). Coherence operations (invalidations, writebacks) produce extra memory traffic and inter-core stalls:

- Write miss/invalidation: writer invalidates other cached copies → other cores observing invalidations may experience cache misses on subsequent accesses.
- Cache line ownership transfer: causes additional latency, modeled as an extra miss penalty or coherence delay  $T_{coh}$ .

These events are data-dependent and create inter-task timing interference.



**Figure:** TLB and multi-level page table flowchart (translation pipeline).

## e. Mathematical Expression for Interference

A more comprehensive WCET formula, for a task  $\tau$  on multicore platform with  $M$  interfering cores, is:

$$WCET_{\tau} = C_{cpu}^{\tau} + N_{mem}^{\tau} \cdot (T_{L1} + MR_{L1}^{wc,\tau} \cdot (T_{L2} + MR_{L2}^{wc,\tau} \cdot (T_{DRAM} + I_{DRAM}))) + I_{coh} + I_{PTW}$$

where:

- $I_{DRAM}$  is worst-case additional DRAM access latency due to contention from other cores,
- $I_{coh}$  aggregates coherence-induced stalls,
- $I_{PTW}$  aggregates worst-case page-table walk overheads and TLB shutdown delays.

Interference can be conservatively upper-bounded by summing worst-case accesses from all other cores scaled by worst-case per-access latency, but such bounds are often overly pessimistic; advanced analyses use per-bank arbitration and phase modeling to tighten estimates.

## 4. Detailed Analytical Discussion

This section provides quantitative, cycle-level examples, shows how miss-rate variations affect WCET, models translation overhead within control loops, analyzes multicore interference mathematically, and contrasts MPU vs MMU.

### a. Numerical Example Single Core

Assume an ARM Cortex-A-class core with the following microarchitectural parameters (representative, rounded):

- $T_{L1} = 1$  cycle,
- $T_{L2} = 12$  cycles,
- $T_{DRAM} = 200$  cycles,
- $T_{PTW} = 300$  cycles,

- $N_{\text{mem}} = 2000$ ,
- $C_{\text{cpu}} = 1,000,000$  cycles.

**Case A:** Optimistic L1 miss rate  $MR_{L1} = 0.02$ ,  $MR_{L2} = 0.005$ , P TLB miss = 0.001.

$$\begin{aligned} \text{AMAT} &= 1 + 0.02 \text{ multiplied by } 12 + 0.005 \text{ multiplied by } 200 \\ &= 1 + 0.02 \text{ multiplied by } 13 = 1 + 0.26 = 1.26 \text{ cycles} \end{aligned}$$

**Translation overhead per access:**

$$T_{\text{trans}} = 0.001 \text{ multiplied by } 300 = 0.3 \text{ cycles}$$

Effective cost per memory reference approximately 1.56 cycles

**WCET approximately:**

$$\begin{aligned} &1,000,000 + 2000 \text{ multiplied by } 1.56 \\ &= 1,003,120 \text{ cycles} \end{aligned}$$

**Case B:** Slightly worse L1 miss rate  $MR_{L1} = 0.06$  (e.g., due to context switch / cache interference),  $MR_{L2} = 0.01$ , P TLB miss = 0.005.

$$\begin{aligned} \text{AMAT} &= 1 + 0.06 \text{ multiplied by } 12 + 0.01 \text{ multiplied by } 200 \\ &= 1 + 0.06 \text{ multiplied by } 14 = 1 + 0.84 = 1.84 \text{ cycles} \end{aligned}$$

**Translation overhead per access:**

$$T_{\text{trans}} = 0.005 \text{ multiplied by } 300 = 1.5 \text{ cycles}$$

Effective cost per memory reference approximately 3.34 cycles

**WCET approximately:**

$$\begin{aligned} &1,000,000 + 2000 \text{ multiplied by } 3.34 \\ &= 1,006,680 \text{ cycles} \end{aligned}$$

**Result:** A change in miss rates from 2% to 6% and TLB miss up to 0.5% increases WCET by  $\approx 3,560$  cycles, but in hard real-time systems even such increases may violate deadlines. It demonstrates that small variations in microarchitectural statistics can materially affect WCET.

## b. Sensitivity to Miss Rate Variation

Different application phases (sensor processing, control update, communication) have distinct access patterns. Let  $\Delta(MR_{L1})$  be the deviation in miss rate from baseline. The sensitivity of WCET to miss-rate variation is:

$$\frac{\partial WCET}{\partial MR_{L1}} = N_{mem} \cdot (T_{L2} + MR_{L2} \cdot T_{DRAM})$$

Thus, per-percentage-point increase in L1 miss rate multiplies by the product  $N_{mem}$  and L2/DRAM penalties, showing why high penalty memory accesses (DRAM) amplify WCET sensitivity.

## c. Translation Overhead in Control Loops

Consider a tight control loop executing  $N_{iter}$  iterations with one memory access per iteration that touches a new page every  $K$  iterations (i.e., page boundary stride). TLB miss rate per access  $P_{TLB\_miss} = 1/K$ . Loop translation overhead:

$$T_{loop\_trans} = N_{iter} \cdot P_{TLB\_miss} \cdot T_{PTW} = N_{iter} \cdot \frac{1}{K} \cdot T_{PTW}$$

If  $K = 64$  (common case with 4 KB pages and 256 B cache lines under a stride pattern),

$T_{PTW} = 300$  cycles, and  $N_{iter} = 1000$ ,

Then,  $T_{loop\_trans} = 4,687.5$  cycles,

which represents a significant contribution relative to the loop body execution time. using TLB locking or large pages reduces  $P_{TLB\_miss}$  and therefore lowers the overall address translation overhead.

## d. Multicore Interference

Model a two-core system:

- Core 1 runs a real-time control task  $\tau$  with  $N_{mem\_t}$  worst-case memory accesses.
- Core 2 is a best-effort task and issues  $N_{mem\_b}$  memory

accesses in the overlapping interval.

Let DRAM arbitration provide a worst-case per-access service time  $T_{DRAM}$ , and assume arbitration latency increases linearly with competing requests. A conservative interference bound for the additional latency  $I_{DRAM}$  per access is:

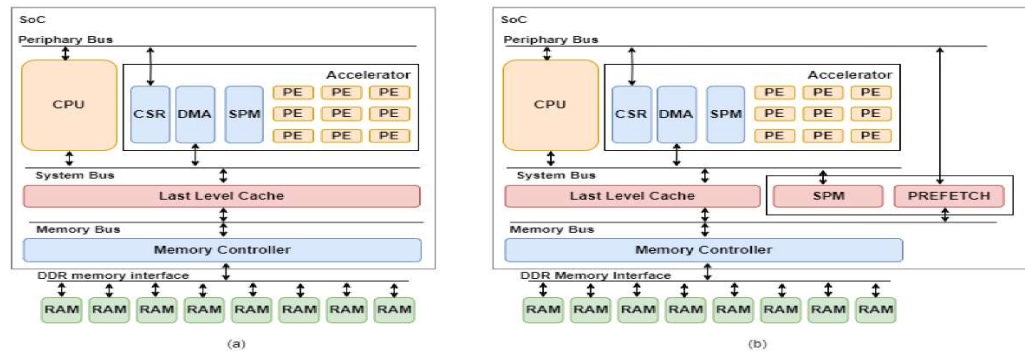
$$I_{DRAM} = \max_{k \in [1, M-1]} (k \cdot T_{DRAM\_slot})$$

A more refined bank-aware model partitions DRAM into  $B$  banks. Accesses to different banks can proceed in parallel, reducing worst-case interference. If the memory controller enforces per-core bandwidth  $BW_i$ , the worst-case DRAM latency for core 1 can be bounded using queuing theory (for example,  $M/G/1$  worst-case bounds) or network-calculus-style latency bounds. Recent work applies static analysis to limit overestimation through phase modeling and static allocation of banks and cache ways.

Coherence adds a multiplicative factor. If every write from core 2 invalidates a cache line used by task  $\tau$ , then the worst-case number of extra misses equals the number of invalidations. Let  $V$  denote the worst-case number of invalidations. The resulting coherence interference is:

$$I_{coh} = V \cdot (T_{coh\_miss})$$

where  $T_{coh\_miss}$  approximates the cache miss penalty plus the coherence transfer cost.



**Figure:** DRAM bank partitioning & mem\_controller QoS schematic.

## **e. MPU Versus MMU Comparison**

MPU provides region-based protection with negligible translation latency and high predictability. MMU provides virtual memory, isolation, and flexibility but introduces TLB misses and page walks.

**Predictability and certification simplicity:** MPU preferable.

**Latency:** MPU lower worst-case latency

**Flexibility:** MMU higher

For safety critical robotics, MPU or restricted MMU configurations with locked mappings are often favored.

## **5. Critical Evaluation**

### **a. Performance Vs Predictability Vs Power**

Caches and MMU improve throughput and energy efficiency but complicate WCET estimation. Removing them improves predictability but may require higher clock frequency or additional hardware to meet computational demands. Designers must balance computational requirements of perception and AI with strict deadlines of motor control.

### **b. Certification Implications**

Certification standards (ISO 26262, DO-178C, IEC 61508) require evidence of deterministic behavior and bounded response times. Systems using MMUs and complex caches face larger verification burdens: proofs or evidence must account for TLB behaviour, page table consistency, and multicore interference. Use of MPUs, static allocation, and partitioned resources simplifies compliance by reducing nondeterministic phenomena. Real-time virtualization and time-predictable MMU mechanisms offer pathways for certification but demand extensive analysis and OS/hypervisor support.

## c. Limitations of Deterministic Techniques

- Cache partitioning: Static partitions can underutilize cache; dynamic partitioning complicates analysis and may reintroduce timing variability.
- Way-locking: Requires hardware support; locking overhead and eviction on context switch can be problematic.
- Scratchpad: Requires static or dynamic allocation strategies; dynamic scratchpad allocation may need complex compiler/runtime support to remain analyzable.
- TLB locking/time-predictable virtual memory: Useful but incomplete when interactions with coherence, interrupts, and DMA are considered.
- Measurement-based WCET: Risk of missing pathological scenarios; must be combined with static guarantees for certification.

## d. Mechanism Comparison

| Mechanism                     | Latency (avg)                | Worst-case analyzability    | Flexibility | Power  | Implementation cost |
|-------------------------------|------------------------------|-----------------------------|-------------|--------|---------------------|
| L1/L2 cache (shared)          | Low                          | Low (hard)                  | High        | Medium | Low                 |
| Cache partitioning (coloring) | Moderate                     | Improved                    | Medium      | Medium | Medium              |
| Way-locking / cache locking   | Moderate                     | High (when used)            | Low         | Medium | High (HW & SW)      |
| Scratchpad memory             | Deterministic (single cycle) | High                        | Low         | Low    | Medium (SW alloc)   |
| MPU (no VM)                   | Low                          | High                        | Low         | Low    | Low                 |
| MMU with TLB locking          | Varied                       | Moderate (with constraints) | High        | Medium | High                |
| DRAM bank partitioning        | Varied                       | Improved                    | Medium      | Medium | Medium              |

## 6. Architectural Strategies and Design Recommendations

- Use heterogeneous architecture separating real time control on Cortex M or isolated core and high-level perception on Cortex A.

- Apply cache partitioning or coloring for isolation.
- Lock critical code and data into cache or place in scratchpad.
- Partition DRAM banks and enforce bandwidth control.
- Lock TLB entries or use static mappings for real time tasks.
- Combine static WCET analysis with measurement under stress conditions.

Such combined strategies balance throughput with deterministic guarantees.

## 7. Conclusion and Future Scope

This assignment has examined, with analytical depth, how memory system mechanisms influence real-time determinism in ARM-based embedded robotic controllers. Caches and multilevel memory hierarchies bring substantial average-case performance benefits but introduce variability via miss phenomena, replacement dynamics, and coherence. Address translation (TLBs and page tables) further increases timing unpredictability through page walks and shutdowns. Multicore shared resources compound these effects through interference across caches, DRAM banks, and interconnects.

Quantitative examples show that modest changes in miss rates or TLB behavior can add thousands of cycles to WCET, potentially violating control deadlines. Consequently, system designers must adopt a layered approach:

- (i) use heterogeneity to isolate hard real-time tasks on simple deterministic cores (MPU, scratchpad),
- (ii) apply cache partitioning/locking and DRAM bank partitioning where high throughput cores coexist with real-time tasks, and
- (iii) employ static memory allocation, TLB locking, and WCET-driven scratchpad allocation to bound worst-case latencies.

Future research directions include:

- Time-predictable memory controllers and DRAM architectures: new

DRAM scheduling algorithms and bank architectures that provide tight real-time guarantees without severe throughput penalties.

- Hardware support for deterministic virtualization: MMU extensions and hypervisor primitives that permit virtual memory features while bounding page walk and TLB behaviors deterministically.
- Co-design of ML workloads and predictable hardware: as robotics integrates more AI, research must develop predictable inference engines (deterministic caching or dedicated accelerators with bounded latency).
- Automated WCET-aware memory management: compiler and OS toolchains that automatically partition cache/banks and allocate scratchpad space to minimize WCET while preserving average performance.

In sum, balancing high performance and determinism in embedded robotics requires combined hardware, OS, and software strategies, careful WCET analysis, and an acceptance of trade-offs between flexibility and certifiability. Continued research targeting DRAM predictability, deterministic virtualization, and AI-aware predictable architectures will be essential to meet the next generation of safety-critical robotic CPS requirements.